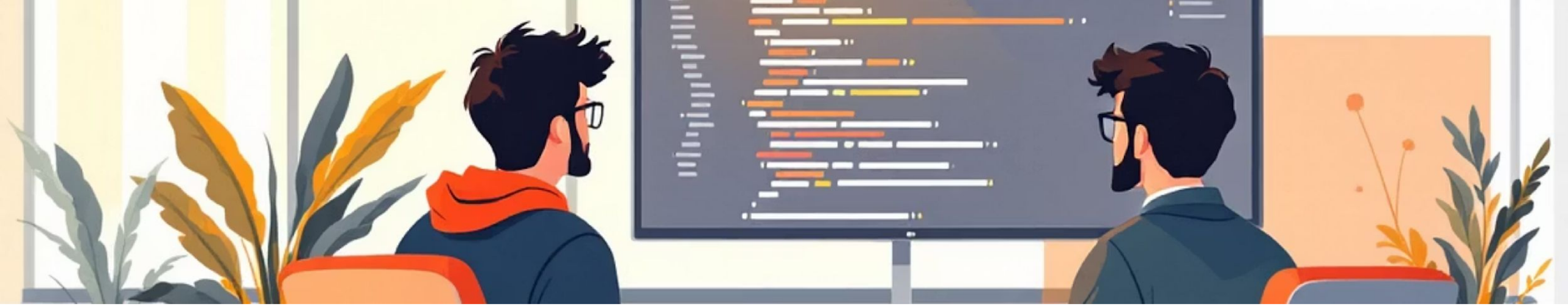
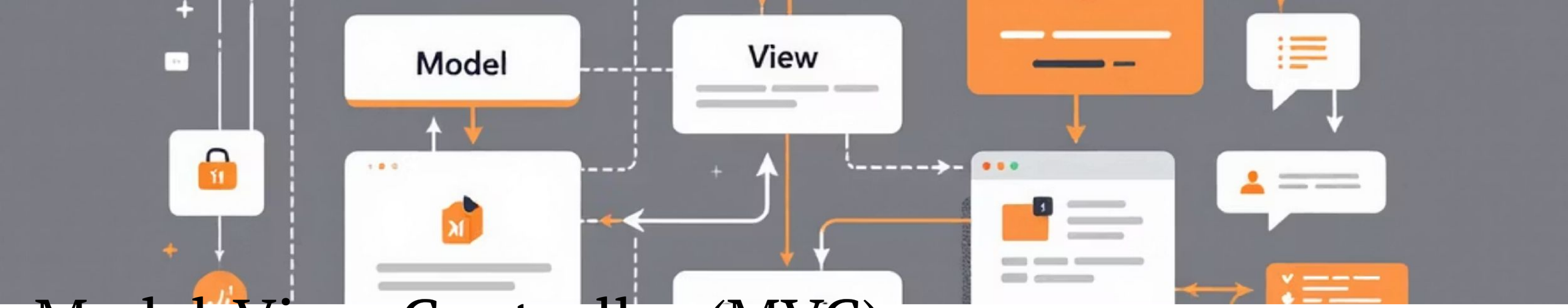
Architectural & Enterprise Patterns: Practical Guide for System Designers

Concise survey of three foundational patterns — MVC, State/Logic Controller, and Sense-Logic-Actuator — with architecture tradeoffs, usage scenarios, and integration guidance for real systems.



Why Patterns Matter

Patterns encode proven structural decisions that improve maintainability, scalability, and team alignment. They reduce ambiguity, accelerate onboarding, and enable predictable evolution of complex systems.



Model-View-Controller (MVC)

MVC separates responsibilities across three roles: Model (data + business rules), View (presentation), Controller (input & orchestration). Common in web frameworks (Rails, ASP.NET) and desktop GUIs.

Model–View–Controller (MVC) is an **architectural pattern** that divides an application into three separate components to improve **modularity, maintainability, and scalability**.

Model

The **Model** represents the **data and business logic** of the application.

Responsibilities

- Stores application data
- Performs business rules and calculations
- Interacts with the database

Example

In an **online shopping system**, the model may include:

- Product data
- Customer information
- Order details

View

The **View** is the **presentation layer (User Interface)**.

Responsibilities

Displays data to the user

Shows forms, buttons, and screens

Receives user input visually

Examples:

Web pages (HTML)

Mobile UI

Desktop interface

Controller

The **Controller** acts as a **bridge between Model and View**.

Responsibilities

- Receives user input
- Processes requests
- Updates the model
- Sends results back to the view

Example:

When a user clicks **“Buy Product”**:

1. Controller receives request
2. Controller updates the order in the model
3. View displays confirmation

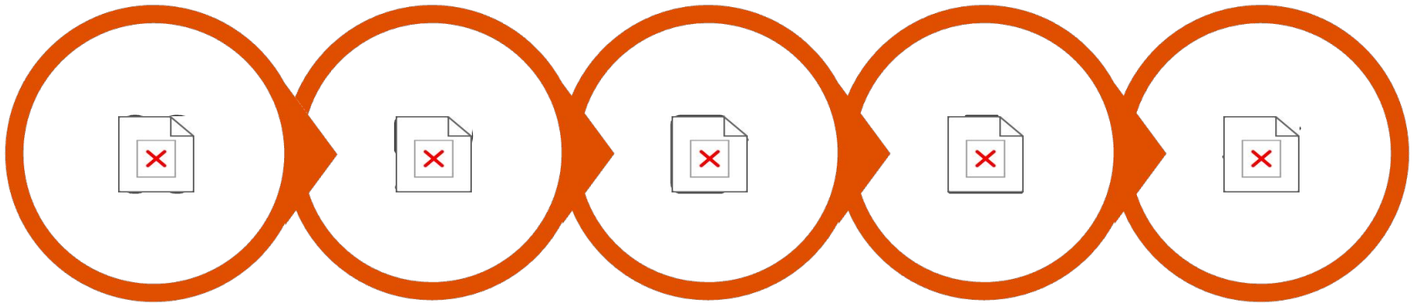
MVC: Key Benefits & Considerations

- **Separation of concerns — easier testing and parallel development**
- **Fine-grained control over URLs and view rendering (SEO, REST)**
- **Supports TDD**(Test Driven Development)**: models and controllers testable independently**
- **Risks: Overly thin controllers(**Controller only calls model methods without much logic) **or fat models(**too much logic is added to the Model, it becomes very complex and difficult to maintain)**; coupling via shared session/state(**components share **session data or global state)**, which creates **tight coupling** between components

State / Logic Controller Pattern

Focuses on managing system behaviour through explicit states and allowed transitions. Ideal where correctness depends on state sequences (wizards, payment flows, device modes).

An architectural pattern where a controller manages system behavior using **explicit states and defined transitions between those states**.



This FSM (Finite State Machine)ensures only valid transitions occur and centralizes validation and rollback logic.

Practical Tradeoffs

- Pros: Explicit invariants, easier reasoning, safer concurrent handling
- Cons: Can be verbose for simple flows; state explosion in complex domains
- Integration: Expose state via events or a state service; keep side effects in controlled handlers

Sense → Logic → Actuator Pattern

A closed feedback loop: Sense (ingest sensor or external data), Logic (decide using rules/models), Actuator (perform changes). Central to IoT, robotics, and real-time automation.

The **Sense–Logic–Actuator (SLA)** pattern is an **architectural pattern** used mainly in **embedded systems, robotics, and IoT devices**.

It divides a system into **three main parts**:

1. **Sense**
2. **Logic**
3. **Actuator**

This pattern allows a system to **observe the environment, process the information, and perform an action**.

Sense (Sensors):

The **Sense component** collects data from the environment.

Examples of sensors : Temperature sensor ,Motion sensor ,Light sensor ,Camera , Microphone

Example:

A **motion sensor** detects movement near an automatic door.

Logic (Processing Unit)

The **Logic component** processes the data received from sensors and decides what action should be taken.

It acts like the **brain of the system**.

Devices used: Microcontroller , Computer processor ,Control system software

Actuator (Action)

The **Actuator** performs the action decided by the logic component.

Examples:Motor,Alarm,Heater,Light,Door opener

Architectural Best Practices

- Design for latency: local decisions at the edge, heavy analytics in cloud
- Use idempotent actuations and command audits logs to handle retries
- Model uncertainty(Real-world systems often have **noise and uncertain data** from sensors.): incorporate thresholds , hysteresis(Prevents systems from **switching on/off repeatedly**.), and safety guards
- Security: authenticate sensors, encrypt telemetry(**data sent from sensors to servers**.), authorize actuator commands(Only **authorized systems** should control actuators.)

Design for Latency

Latency means the **delay between sending a request and receiving a response**.

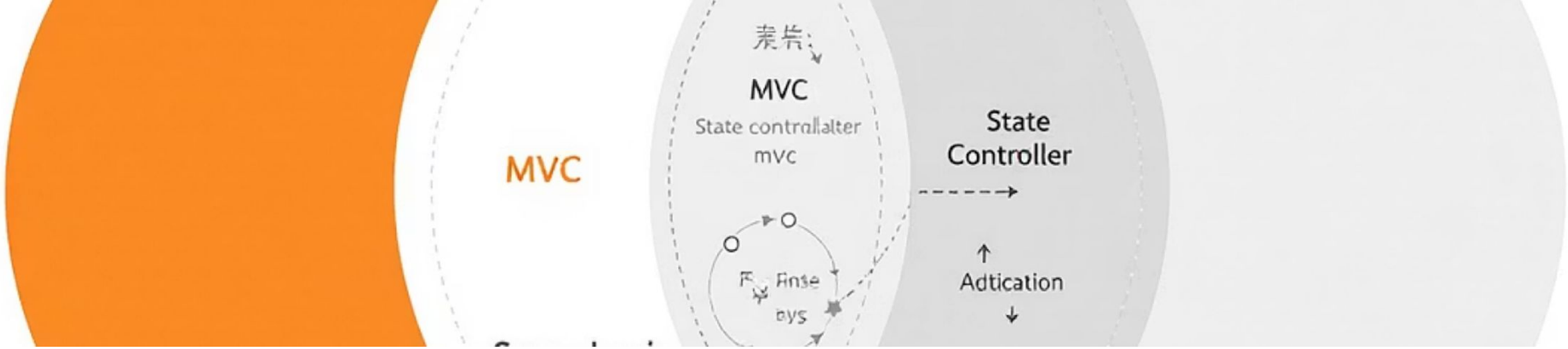
In modern architectures:

- **Edge devices** make **quick local decisions**.
- **Cloud systems** perform **heavy analytics and data processing**.

Example

Smart Traffic System

- Camera (edge device) detects traffic → immediately changes signal.
- Cloud server analyzes **long-term traffic patterns**.



How These Patterns Work Together

Examples: a web UI uses MVC for presentation while a backend state controller manages order workflow; connected devices use Sense-Logic-Actuator to feed data that updates models within the MVC backend.

Design principle: pick the pattern that addresses the primary concern (UI, state correctness, or feedback loop) and compose—don’t force a single pattern to solve everything.

Decision Checklist & Next Steps

Identify Primary Concern

Is the system UI-centric, state-critical, or real-time reactive?

Map Boundaries

Assign responsibilities: UI, orchestration, sensing, actuation, persistence.

Specify Contracts

Define events, state transitions, APIs, and failure modes before implementation.

Safety & Observability

Design guards, retries, logging, metrics, and end-to-end tests.

Ready to prototype: pick one user story, model it with the chosen pattern, and validate with tests and telemetry.